Connecting a Custom Domain to Firebase App Hosting

Fulllion Creative Works

A step-by-step guide to publishing a website on **Firebase App Hosting** under your own domain. It starts from a domain registered at **Squarespace**, moves its DNS to **Cloudflare**, puts your code on **GitHub**, and ends with a live site on `https://example.com` (and, if you want one, a second address like `https://app.example.com`) with sign-in working.

It is written for someone who has never done this before. Every step says where to click, what you should see, and what to do when you don't see it. The snags that cost hours the first time are written in where they happen, and gathered again in Troubleshooting.

> **Building the app itself?** This guide starts from a working app. For setting up a Firebase app, the emulators, security rules and the App Hosting basics, see the companion guide, Building Firebase Apps in Google Antigravity IDE.

> **Storing users' files?** Once your site is live, Private Cloud Storage for a Next.js Site on Firebase App Hosting adds a private bucket with signed upload and download links.

Throughout, replace:

Placeholder	With
`example.com`	your domain
`app.example.com`	a second address on the same site, if you want one (skip those steps if not)
`your-project-id`	your Firebase project's ID

Contents

1. How the Pieces Fit Together
2. Move Your DNS from Squarespace to Cloudflare
3. Set Up Cloudflare: HTTPS, Encryption and the www Redirect
4. Create and Prepare the Firebase Project
5. Put Your Code on GitHub
6. Create the App Hosting Backend
7. Connect Your Custom Domains
8. Test the Live Site
9. Two Addresses, One Backend: X-Forwarded-Host
10. Deploying Changes from Now On

11. GitHub Actions on a Private Repository
12. Signing In the GitHub Command Line
13. Troubleshooting
14. Sources

Before You Start

You'll need:

- **A domain** you've registered. This guide uses Squarespace (which took over Google Domains); other registrars work the same way, with their own menu names.
- **A website that builds.** A Next.js app in a folder on your computer that runs `npm run build` without errors, tracked with Git.
- **Accounts:** Cloudflare (the free plan is enough), a Google account for Firebase, and GitHub.
- **GitHub Desktop** installed and signed in.
- **Optional: Google Antigravity IDE,** if you also want to release from your own computer, not only by pushing to GitHub (Section 10).
- **A payment card.** Firebase App Hosting needs Firebase's pay-as-you-go plan (Blaze). A small site usually costs little, and Section 4 sets a budget alert.
- **About two to three hours**, most of it waiting for DNS and certificates. You can stop at any of the waits and come back.

The order matters, so follow the sections in turn:

1. **DNS first**, because changing nameservers can take a while to settle, and it settles while you do the rest.
2. **Firebase next**, since App Hosting needs the Blaze plan before it will create anything.
3. **GitHub before App Hosting**, because App Hosting builds your site from a GitHub repository.
4. **Domains last**, because Firebase can only hand you the records once the backend exists.

---

1. How the Pieces Fit Together

If these words are already familiar, skip to Section 2.

Piece	What it does	In this guide
Registrar	Where you rent the domain. Renewals happen here, and it says which nameservers answer for the domain.	Squarespace
Nameservers / DNS host	The servers that answer "where is example.com?". You manage your **DNS records** here.	Cloudflare
Hosting	The servers that run your site and answer the visitor's browser.	Firebase App Hosting

Piece	What it does	In this guide
Code host	Where your code lives. App Hosting builds from it (or from your own folder; see Section 10).	GitHub

When someone visits `https://example.com`:

```
Browser ──"where is example.com?"──▶ Cloudflare's nameservers ──"at this address"──▶ Browser
Browser ──────────────── visits that address ────────────────▶ Firebase App Hosting
```

DNS records are the lines in that answer:

Type	Means	Example
A	"this name is at this IPv4 address"	`example.com → 203.0.113.10`
AAAA	the same, for an IPv6 address	`www → 100::`
CNAME	"this name is another name; ask about that one"	`_acme-challenge_abc → abc.authorize.certificatemanager.goog`
TXT	a line of text, often proof that you own the domain	`fah-claim=…, v=spf1 -all`
MX	where email for the domain goes	(none, if the domain has no email)

In Cloudflare, each record also has a **proxy status**:

- **Orange cloud, "Proxied":** visitors reach Cloudflare, and Cloudflare passes the request on (or answers it itself, as with the `www` redirect in Section 3).
- **Grey cloud, "DNS only":** Cloudflare only gives out the address, and visitors go straight to your host. **Every Firebase record in this guide is grey.**

Caching: a DNS answer, including "there's no such record", is remembered by everyone who asked, for a while. Cloudflare's default for a "no such record" answer is **30 minutes**. That's why something you just added can look missing for a while (see Section 7.4).

---

2. Move Your DNS from Squarespace to Cloudflare

You keep the domain registered at Squarespace and keep paying for it there. Only the job of answering DNS questions moves to Cloudflare.

2.1 Add the domain to Cloudflare

1. Sign in at dash.cloudflare.com, open **Domains**, and choose **Onboard a domain** (older dashboards say **Add a domain**).
2. Type your domain, `example.com`, with nothing before it: no `www`, no `https://`.
3. Leave the **quick scan for DNS records** on, and continue. Cloudflare says the scan may not find every record. If you know of one it missed (an email record, say), add it by hand before activating.
4. Choose the **Free** plan.

5. Cloudflare now shows the DNS records it found at Squarespace. **Don't press "Continue to activation" yet**: tidy them first (2.2).

2.2 Tidy the imported records

A Squarespace domain comes with records pointing at Squarespace's own website servers. Since your site will be on Firebase, remove those, and keep anything to do with email.

Delete (these usually have the orange cloud):

Record	Why it's there
Four **A** records for `@` (`198.185.159.144`, `198.185.159.145`, `198.49.23.144`, `198.49.23.145`)	Squarespace's web servers
CNAME `www` → `ext-sq.squarespace.com`	the same, for `www`
CNAME `_domainconnect`	lets Squarespace set up other services by itself; not needed once Cloudflare holds your DNS

Keep (these are grey, "DNS only"):

Record	What it does
TXT `v=spf1 -all` on `@`	tells other mail servers that nothing sends email from your domain
TXT `_dmarc` (`v=DMARC1; p=reject; …`)	tells them to reject mail that claims to be from your domain
TXT `_domainkey` (`v=DKIM1; p=`)	the same idea, for signed mail
MX records, if any	where your domain's email goes. Keep them if you use email at this domain.

Together, the three TXT records stop anyone sending fake email "from" your domain when it has no email of its own. If you later add real email, they need changing in the same go (see Troubleshooting).

> **Not sure what a record is?** Leave it, finish the guide, and come back to it. Deleting an email record you need breaks your email; leaving an unused one breaks nothing.

2.3 Turn off DNSSEC at Squarespace

DNSSEC signs your DNS answers. The signatures belong to Squarespace's nameservers, so if DNSSEC is still on when you switch to Cloudflare's, the internet sees answers whose signatures don't match, and your domain can **stop working entirely for a day or more**.

1. Go to your Squarespace **Domains** dashboard and open the domain.
2. Open **DNS**, then **DNSSEC**.
3. If **DNS Security Extensions** is on, switch it off and **Confirm**. If it was already off, carry on.

Squarespace says it turns DNSSEC off by itself when you switch to custom nameservers (2.4 shows the prompt), so this step is a safety check. Cloudflare's own setup page asks for it too ("Make sure DNSSEC is off"), and it costs nothing to check. You can turn DNSSEC back on later, through Cloudflare, once

everything works.

2.4 Switch Squarespace to Cloudflare's nameservers

Back in Cloudflare, press **Continue to activation**. Cloudflare shows **two nameservers**, for example `ada.ns.cloudflare.com` and `bob.ns.cloudflare.com`. Yours will have different names; use the ones Cloudflare shows you.

Then in Squarespace:

1. Open the domain, then **DNS**, then **Domain Nameservers**.
2. Choose **Use Custom Nameservers**. Squarespace asks for your password again (and your two-step code, if you use one), then shows a note about DNSSEC. Press **Continue** at each.
3. Enter Cloudflare's two nameservers as **Nameserver 1** and **Nameserver 2**. Remove Squarespace's own four (`nsb1` to `nsb4.squarespacedns.com`) if they're still listed.
4. **Save.** Squarespace shows "Nameservers updated", and says the change can take up to 48 hours. It also warns that Squarespace's own DNS records stop applying and some of its domain features stop working. That's expected: Cloudflare holds your records now.

> **Wrong place:** Squarespace also has a page called **Nameserver registration** with a **Create a host record** button. That's for running *your own* nameservers (like `ns1.example.com`, called glue records). You don't want it. Close it without saving.

2.5 Wait for "Active", and check it

The internet usually notices the change within an hour, sometimes much faster. Until Cloudflare does, the domain's status is **Pending Nameserver Update**. Cloudflare checks by itself every so often, and emails you when the domain is **Active**. The Overview page then says **"Your domain is now protected by Cloudflare"**. A free-plan domain that stays pending for 28 days is removed from Cloudflare, so don't leave this step half-done.

You can check from your own computer at any time. In PowerShell:

```
Resolve-DnsName example.com -Type NS -Server 1.1.1.1
Resolve-DnsName example.com -Type NS -Server 8.8.8.8
```

Both should list Cloudflare's two nameservers. On macOS or Linux:

```
dig +short NS example.com @1.1.1.1
```

> **No "Check nameservers now" button?** Some versions of the dashboard don't show one. Just wait for the email or the Overview page to change.

From now on, every DNS change happens in Cloudflare. Squarespace's DNS page is ignored while your nameservers point at Cloudflare. Records added there do nothing.

---

3. Set Up Cloudflare: HTTPS, Encryption and the www Redirect

3.1 Always Use HTTPS

1. In Cloudflare, open your domain, then **SSL/TLS**, then **Edge Certificates**.
2. Turn on **Always Use HTTPS**.

This sends `http://` visitors to `https://`. It only affects orange-cloud records; Firebase handles HTTPS itself for the grey ones. If you can't find the switch, the encryption mode (3.2) is set to Off, which hides it.

3.2 The encryption mode

When a record has the orange cloud, a visit makes two connections:

1. **Visitor to Cloudflare:** always HTTPS, using Cloudflare's free certificate.
2. **Cloudflare to your host:** the **encryption mode** decides how this second leg works.

Mode	Cloudflare to your host	Use it?
Off / **Flexible**	plain HTTP, unencrypted	**No.** With a host that sends HTTP to HTTPS, like Firebase, this causes an endless "too many redirects" loop. It's a classic mistake.
Full	HTTPS, any certificate accepted	Fine
Full (strict)	HTTPS, and the certificate must be valid	Best, when your host's certificate is valid, as Firebase's is
Automatic SSL/TLS (Cloudflare's suggestion)	Cloudflare tests your host, picks the most secure mode that works, and only ever moves up	Fine

There's also **Strict (SSL-Only Origin Pull)**, which is for paid plans and can be ignored.

To see or change it: **SSL/TLS**, then **Overview**. In the newer dashboard, the **Configure** button at the top right offers **Automatic SSL/TLS** or **Custom SSL/TLS**; under Custom you pick the mode. Some accounts haven't been moved to Automatic yet, and see only the list of modes. It is not on the Edge Certificates page.

In this guide **no orange-cloud record points at a real server** (`www` points nowhere; see 3.3), so the mode makes no difference today. Full, Full (strict) or Automatic are all good choices. Avoid Flexible.

3.3 Redirect www to your domain

Visitors often type `www.`. Cloudflare can send them on to `example.com` itself, so Firebase never needs to know about `www`.

Part 1: a stand-in record. Open **DNS**, then **Records**, then **Add record**:

Field	Value
Type	`AAAA`
Name	`www`

Field	Value
IPv6 address	`100::`
Proxy status	**Proxied** (orange cloud)
TTL	Auto

`100::` is a special address that leads nowhere. Cloudflare answers for `www` itself, so it never needs a real server behind it. This is Cloudflare's own recommendation for a name with no server; an `A` record to `192.0.2.0`, proxied, does the same.

Part 2: the redirect rule.

1. Open **Rules**, then **Overview**, then **Create rule**, and choose **Redirect Rule**.
2. Pick the template **Redirect from WWW to Root**. It fills in:
   - **If incoming requests match:** Wildcard pattern, **Request URL** `https://www.*`
   - **Then: Target URL** `https://${1}`, status **301**
3. Tick **Preserve query string**, so a link like `www.example.com/sign-in?next=app` keeps the part after `?`.
4. Press **Deploy**.

If Cloudflare warns **"This rule may not apply to your traffic"**, check that the `www` record from Part 1 is there and orange. If it is, choose **Ignore and deploy rule anyway**: the warning can show while the domain is still pending.

Test it once the domain is Active. In PowerShell, use `curl.exe`, not `curl`, which in Windows PowerShell is a different command:

```
curl.exe -sI "https://www.example.com/pricing?x=1"
curl.exe -sI "http://www.example.com/pricing"
```

The first should answer `301 Moved Permanently`, with `Location: https://example.com/pricing?x=1`. The second should give a 301 to `https://www.example.com/pricing`, and from there the rule takes over.

> **Right after the domain turns Active,** `www` can show as "doesn't exist" for up to half an hour, even with the record correct. Cloudflare is still switching the newly active domain on. Wait, then test again.
>
> `http://www…` **gives error 522?** Always Use HTTPS (3.1) isn't on yet: the rule only matches `https://`.

---

4. Create and Prepare the Firebase Project

4.1 Create the project

1. Go to console.firebase.google.com and choose **Create a project**.
2. **Name** it, accept the terms, and continue.

3. **The project ID** appears under the name. It's permanent, and visitors see it: Google's sign-in window says "*to continue to **your-project-id**.firebaseapp.com*". Firebase may add characters to make it unique (like `mysite-1a2b3`). Edit it now if you want a cleaner one; you can't later.
4. Two optional steps follow: **Gemini in Firebase**, and **Set up Google Analytics**. Skip Analytics if you don't want analytics on your site. You can add it later, under **Project settings**, **Integrations**.
5. Press **Create project**.

4.2 Upgrade to the Blaze plan, and set a budget

App Hosting needs a project on the pay-as-you-go Blaze plan, and won't create anything on the free Spark plan. Blaze still includes no-cost allowances (for example, some bandwidth each month), so a small site with little traffic usually costs little.

1. In the console, choose **Upgrade** (near the plan name, "Spark", at the bottom of the left menu) and pick **Blaze**.
2. Create or choose a Google Cloud **billing account**, and add your card.
3. **Set a budget alert**, for example $10 a month. The upgrade offers one. To add one later: **Settings**, then **Usage and billing**, then **Details & settings**, then **Budget alerts**, which opens Google Cloud's **Budgets & alerts**.

A budget alert **emails you**; it doesn't stop the spending. If you want a hard stop, Firebase also offers **spend caps** (in preview at the time of writing), which can pause App Hosting when a budget is reached. Look for **Service-level spend caps** on the same page. A paused site is offline, so choose its level with care.

4.3 Turn on sign-in

If your site has accounts:

1. Open **Security**, then **Authentication**, and press **Get started**.
2. **Sign-in method** tab:
   - **Email/Password:** enable it and **Save**. Inside it, **Email link (passwordless sign-in)** is a second switch: turn it on only if your site offers sign-in links by email.
   - **Google:** enable it. It asks for a **support email** to show in the sign-in window.
3. **Settings** tab, then **Authorized domains:** press **Add domain** and add `example.com`. Google sign-in, and links in sign-in emails, won't work on your domain without it.
   - `localhost` **isn't authorized by default** in projects created since April 2025. Add it too if you sign in while testing on your own computer.
   - You only need the addresses where people sign in. A second address that never shows a sign-in form doesn't need adding.

For Google sign-in on your own domain, a **pop-up** sign-in (`signInWithPopup` in Firebase's web SDK) works without extra setup. A **redirect** sign-in on a custom domain needs extra steps; see Firebase's redirect best-practices page in Sources.

Sign-in links (no password)

A sign-in link lets someone type only their email. Firebase emails them a one-time link, and opening it signs them in. To offer it:

1. In **Security**, **Authentication**, **Sign-in method**, open **Email/Password**.
2. Make sure **Email/Password** itself is enabled: sign-in links need it.
3. Turn on **Email link (passwordless sign-in)** too, and **Save**.
4. Check that your domain is in **Authorized domains** (step 3 above). The link brings people back to a page on your site, and Firebase only sends them to an authorized domain.

In your code, with Firebase's web SDK, it has two halves:

```
import { sendSignInLinkToEmail, isSignInWithEmailLink, signInWithEmailLink } from 'firebase/auth';

// 1. Send the link. The address it returns to must be on an authorized domain.
await sendSignInLinkToEmail(auth, email, {
  url: 'https://example.com/sign-in',
  handleCodeInApp: true, // must be true: the sign-in finishes on your page
});
localStorage.setItem('emailForSignIn', email); // so this browser needn't ask again

// 2. Back on /sign-in, when the visitor opens the link:
if (isSignInWithEmailLink(auth, window.location.href)) {
  let email = localStorage.getItem('emailForSignIn');
  if (!email) email = askTheVisitorForTheirEmail(); // opened on another device
  await signInWithEmailLink(auth, email, window.location.href);
  localStorage.removeItem('emailForSignIn');
}
```

Points from Firebase's page:

- **Never put the email in the link's address** to skip asking for it. Firebase warns that this can let someone else's session be slipped in. Keep it in the browser, as above, and ask when it isn't there.
- **A link sign-in confirms the email address,** like Google sign-in does.
- **Completing it tidies the account:** if the same email had an unconfirmed password, that password is removed, and older sessions on the account end.
- **Links expire and work once.** An old or used link gives an "invalid" or "expired" error; send a new one.
- **Web sites are unaffected by the Dynamic Links shutdown** (August 2025), which changed sign-in links for mobile apps only. Leave out the old `dynamicLinkDomain` setting. `linkDomain` is only needed if you've set up custom link domains on Firebase Hosting.

Firebase sends account emails (confirming an address, resetting a password) from `noreply@your-project-id.firebaseapp.com`. Tell people to check their spam folder the first time.

4.4 Create the database

If your site uses Cloud Firestore:

1. Open **Databases & Storage**, then **Firestore**. Some consoles show it under **Product categories**, with a **NoSQL** step on the way.
2. **Create database.** If it asks for an edition, keep the default (Standard).
3. Choose **production mode**. It starts by refusing every read and write from browsers, which is the safe start. Your own rules open what you need.
4. **Choose a location near you and your visitors.** It can't be changed later.

Production mode's default rules stay until you deploy your own with the Firebase CLI (`firebase deploy --only firestore:rules`). App Hosting rollouts don't deploy rules; see the companion guide's rules sections.

4.5 Register a web app, and put its config in apphosting.yaml

1. Open **Project settings** (the gear icon), then **General**, then **Your apps**, and choose the **Web** icon (`</>`).
2. Give it a nickname. **Don't** tick "Also set up Firebase Hosting": you're using App Hosting, which is a different product.
3. **Register app.** Firebase shows a `firebaseConfig` like this:

```
const firebaseConfig = {
  apiKey: "AIza…",
  authDomain: "your-project-id.firebaseapp.com",
  projectId: "your-project-id",
  storageBucket: "your-project-id.firebasestorage.app",
  messagingSenderId: "123456789012",
  appId: "1:123456789012:web:abcdef123456"
};
```

This config is not a secret. Firebase's own words are that it "is considered public": it reaches every visitor's browser anyway. What protects your data is your security rules and your server code (and, if you add it, App Check), not hiding these values. It's normal to commit them. For extra care, Google Cloud lets you restrict the API key to Firebase's own APIs.

If the config includes a `measurementId`, that's Google Analytics. Leave it out if you don't want analytics.

A common way to give these to a Next.js app on App Hosting is the `env` section of **`apphosting.yaml`**, at the root of your repository:

```
runConfig:
  maxInstances: 1

env:
  - variable: NEXT_PUBLIC_FIREBASE_API_KEY
    value: AIza…
  - variable: NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN
    value: your-project-id.firebaseapp.com
  - variable: NEXT_PUBLIC_FIREBASE_PROJECT_ID
    value: your-project-id
  - variable: NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET
    value: your-project-id.firebasestorage.app
  - variable: NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID
    value: "123456789012"
  - variable: NEXT_PUBLIC_FIREBASE_APP_ID
    value: 1:123456789012:web:abcdef123456
```

- The `NEXT_PUBLIC_` prefix tells Next.js to put a value into the browser's code at build time. Your code then reads it as `process.env.NEXT_PUBLIC_FIREBASE_API_KEY`, written out whole.
- Put the number in quotes, as shown, so YAML keeps it as text.
- **Real secrets never go in this file as values.** Keys for a payment provider or webhook secrets go in Cloud Secret Manager, and `apphosting.yaml` refers to them with `secret:` instead of `value:`.
- Each entry can also say when it's needed, with `availability:` `BUILD` and/or `RUNTIME`. Without it, both.
- **Another way:** App Hosting also gives your build the linked web app's config automatically, as `FIREBASE_WEBAPP_CONFIG`. Some setups read that instead. The explicit `NEXT_PUBLIC_` values above are the plainest to follow.

---

5. Put Your Code on GitHub

App Hosting builds your site from a GitHub repository, so your code needs to be on GitHub, on the branch App Hosting will watch: usually `main`.

5.1 Make sure nothing secret is in it

Before publishing, check that your `.gitignore` keeps out anything private. For example:

```
# secrets and local settings: never committed
.env
.env.*
*.pem
*.key
service-account*.json
*-adminsdk-*.json

# dependencies and builds
node_modules/
.next/
```

A Firebase **service account key**, a JSON file with `"type": "service_account"` in it, is a full-power credential. It must never be in a repository. App Hosting doesn't need one: it runs as its own Google account.

5.2 Publish the folder you already have

> **Common mistake:** creating a *new, empty* repository in GitHub Desktop, in a different folder. Your code isn't in it. Publish the folder your code is already in.

1. In GitHub Desktop: **File**, then **Add local repository**, and choose your project's folder.
   - If it says the folder isn't a Git repository, choose **create a repository** there instead.
2. **Is your work on `main`?** At the top, **Current branch** shows the branch.
   - If your work is on another branch, switch to **main**, then choose **Branch**, then **Merge into current branch**, pick your branch, and merge. If `main` has nothing of its own since, this is a simple **fast-forward**, with nothing to resolve.
3. The **Changes** tab may show "*no local changes*". That's fine: publishing sends your **commits** (the History tab), not loose files.
4. Press **Publish repository** at the top. Give it a name, keep **Keep this code private** ticked if you want it private, and publish.

From now on, after each commit, the same button says **Push origin**.

---

6. Create the App Hosting Backend

A **backend** is App Hosting's name for one deployed site, built from one repository and branch.

Before you start, make sure your repository has a **lock file** (`package-lock.json` for npm) committed. App Hosting builds with `npm run build`, and the build fails without one.

1. In the Firebase console, open **Hosting & Serverless**, then **App Hosting**, and press **Get started**, or **Create backend**. If it asks you to upgrade first, that's the Blaze plan from 4.2.
2. **Choose a primary region:** one near your Firestore location.

3. **Connect to GitHub:**
   - A GitHub window opens; allow pop-ups for the Firebase console if nothing appears.
   - GitHub asks you to **install the Firebase GitHub app**. Choose **Only select repositories**, pick your repository, and press **Install**, or **Install & Authorize**.
   - Firebase may ask to give its connection service (Developer Connect) a Secret Manager role, to keep the GitHub connection's token. Allow it.
   - Back in Firebase, choose the repository from the list. If it isn't there, the link beside the list opens the GitHub app's settings, where you can add it.
4. **Deployment settings:**
   - **Root directory:** `/`, unless your app lives in a subfolder of the repository.
   - **Live branch:** `main`.
   - **Automatic rollouts:** on, as it is by default. Every push to `main` then deploys by itself. You can turn this off later, if you'd rather choose when to release (Section 10.3).
5. **Name your backend:** a short name, for example `my-site`. It becomes part of the backend's own address.
6. **Runtime environment:** the newest Node.js is chosen for you. Keep it, or pick the version you build with. Automatic base image updates let Google patch the system under your app by itself.
7. **Web app:** link the web app from 4.5, rather than creating a new one.
8. **Environment variables and secrets,** if the screen offers them: leave both empty if your `apphosting.yaml` already has the values (4.5). Two copies only drift apart.
9. Press **Finish and deploy**.

The console may show these steps in a slightly different order or wording; the choices are the same.

The first **rollout** builds and starts your site. It usually takes 5 to 10 minutes; watch it in the backend's **Rollouts** tab. App Hosting also reports each rollout on GitHub, as a check beside the commit.

If the rollout succeeds, the backend has its own address:

```
https://my-site--your-project-id.REGION.hosted.app
```

Open it: that's your live site, before the custom domain. Test it now (Section 8's checks work on it too).

If the rollout fails, open it and follow its link to the **build log**. The error is near the end, in red. A build that fails on App Hosting usually fails the same way on your own computer: run `npm ci`, then `npm run build`, there first.

---

7. Connect Your Custom Domains

7.1 Add each domain in Firebase

1. In your backend, open the **Settings** tab, then **Add custom domain**.

2. Type `example.com`.
3. If there's a box to **redirect** to another domain, leave it **unticked**. That's only for a domain that should forward somewhere else. You've already handled `www` in Cloudflare (3.3).
4. Continue, and keep **Connect a new domain** selected.
   - **Migrate a domain** is for a domain that's already serving a live site somewhere else. It moves it without downtime, in two steps (prepare the domain, then direct it to App Hosting).
   - A domain with nothing on it yet, like ours after Section 2, is a new domain.
5. Repeat for `app.example.com`, if you want a second address. Section 9 shows how your code tells the two apart.

Each domain now shows **Needs setup**, with a list of records.

7.2 The records, and how they go into Cloudflare

For each domain, Firebase usually gives three records:

Firebase shows	What it's for
A `example.com` → an IP address	where Firebase's servers are
TXT `example.com` → `fah-claim=…`	proves you own the domain
CNAME `_acme-challenge_….example.com` → `….authorize.certificatemanager.goog`	lets Google issue the HTTPS certificate

In Cloudflare, add each one under **DNS**, then **Records**, then **Add record**:

Field	What to enter
Type	as Firebase says
Name	`@` for your bare domain; `app` for `app.example.com`; for the CNAME, everything **before** `.example.com` (for example `_acme-challenge_a1b2c3d4e5f6`)
Content / Target / Value	copied exactly from Firebase, **using its copy buttons**. One wrong character and it won't work.
Proxy status	**DNS only** (grey cloud), for every one
TTL	Auto

Why grey: an orange-cloud record answers with Cloudflare's addresses instead of Firebase's. Firebase's check then sees the wrong address, and can't prove the domain or issue its certificate. (Firebase's own pages don't mention Cloudflare's proxy, but this is what happens.)

Some details:

- **Cloudflare adds `.example.com` to the Name itself.** If you paste the full name, it usually trims it. Check the list afterwards for a doubled name, like `app.example.com.example.com`.
- **The same CNAME for both domains?** Firebase may give your bare domain and your subdomain the **identical** `_acme-challenge` record, because one certificate can cover both. Add it **once**.

- **Several TXT records on one name are fine.** Your new `fah-claim` TXT sits beside your `v=spf1 -all` record. Keep both.
- **A trailing dot** on the CNAME's target (`….goog.`) is fine either way.
- **Leave the TXT and the `_acme-challenge` CNAME in place** after verification. Firebase checks the TXT again later, and needs the CNAME every time the certificate renews.
- **One address per name.** Firebase's docs say to remove any **AAAA** record, and any other A or CNAME record, on a name you connect. After Section 2 there should be none on `@`, but check.

Example, for both domains:

Type	Name	Content	Proxy
A	`@`	the IP Firebase shows	DNS only
TXT	`@`	`fah-claim=…` (from the `example.com` screen)	DNS only
A	`app`	the IP Firebase shows	DNS only
TXT	`app`	`fah-claim=…` (from the `app.example.com` screen)	DNS only
CNAME	`_acme-challenge_…`	`….authorize.certificatemanager.goog`	DNS only

If Firebase lists any records to **remove**, read them before removing anything. Never remove your email records (2.2) because of this step.

7.3 Verify

Back in Firebase, press **Verify records** for each domain. Records Firebase has seen disappear from its list.

7.4 "DNS changes not yet detected"

Firebase often sees some records at once (typically the TXT) and others late (the A and the CNAME). The likely reason: Firebase looked those names up the moment you added the domain, before the records existed, and the "nothing here" answer is remembered for **30 minutes** (see the caching note in Section 1).

Check that your records are right, from your own computer:

```
Resolve-DnsName example.com -Type A -Server 1.1.1.1
Resolve-DnsName app.example.com -Type A -Server 8.8.8.8
Resolve-DnsName example.com -Type TXT -Server 1.1.1.1
Resolve-DnsName _acme-challenge_a1b2c3d4e5f6.example.com -Type CNAME -Server 1.1.1.1
```

If they answer with Firebase's values, **there's nothing to fix: wait.** Pressing Verify again won't speed it up. If a record is still missing in Firebase **an hour** after it answers correctly everywhere, remove the domain in Firebase and add it again, so it asks afresh.

7.5 The certificate

Each domain moves through **Needs setup**, **Pending** and **Minting Certificate**, to **Connected**. Minting usually takes up to a few hours, and Firebase says it can take up to 24. Once it says **Connected**, `https://example.com` serves your site.

> **CAA records:** if your domain has any CAA records (they say which companies may issue certificates for it), they must allow both `letsencrypt.org` and `pki.goog`, or Firebase can't get a certificate. If you have none, any company may, and there's nothing to do.

---

8. Test the Live Site

8.1 From the command line

In PowerShell, again with `curl.exe`:

```
curl.exe -sI https://example.com/
curl.exe -sI http://example.com/
curl.exe -sI "https://www.example.com/about?x=1"
curl.exe -sI https://app.example.com/
```

Request	Should answer
`https://example.com/`	`200`
`http://example.com/`	`301`, to `https://example.com/` (Firebase does this)
`https://www.example.com/about?x=1`	`301`, to `https://example.com/about?x=1` (Cloudflare does this)
`https://app.example.com/`	whatever your app serves there (see Section 9)

A browser's **private window** is a good second test: it has no cached pages or sign-ins.

8.2 Sign in for real

1. Open `https://example.com`, wherever your sign-in page is, and **create an account**.
   - With email and password: open the confirmation email (from `noreply@your-project-id.firebaseapp.com`; check spam), click its link, then sign out and sign in again. A sign-in made *before* confirming still carries "not confirmed".
   - With Google: the email counts as confirmed straight away. If the Google window says the domain isn't authorized, add it under Authentication's **Authorized domains** (4.3).
2. Check that your site shows you as signed in.

If your server code uses Firebase Admin (for example to make session cookies, or read Firestore on the server), App Hosting runs it as its own Google account, without any key file. If signing in works in the browser but the server still treats you as signed out, look in the backend's logs for a **permission** error. The usual fix is giving App Hosting's account (`firebase-app-hosting-compute@your-project-id.iam.gserviceaccount.com`) the roles it's missing in Google Cloud's **IAM** page, such as **Firebase Authentication Admin** or **Cloud Datastore User**. Give only what the error names.

---

9. Two Addresses, One Backend: X-Forwarded-Host

Skip this section if your site has only one address.

One backend can answer several domains, for example `example.com` for the site and `app.example.com` for a separate part of it. Your code has to tell them apart, and on App Hosting there's a catch:

> **On App Hosting, the `Host` header is App Hosting's own address, not your domain.** The domain the visitor typed arrives in **`X-Forwarded-Host`**. On your own computer, a dev server sends your domain in `Host`. So code that reads `Host` alone works locally and then, live, never recognises your second domain: every address gets the main site.

A Next.js example that sends `app.example.com` to its own part of the app. In Next.js 16 the file is `proxy.ts` in `src/` or the project root; before 16 it was `middleware.ts`, exporting `middleware`:

```
import { NextResponse, type NextRequest } from 'next/server';

// The address the visitor asked for: X-Forwarded-Host on App Hosting, Host on a dev server.
// If X-Forwarded-Host holds several entries, the LAST is the one App Hosting added;
// an earlier one could have been sent by the visitor.
function visitorHost(request: NextRequest): string {
  const forwarded = (request.headers.get('x-forwarded-host') ?? '')
    .split(',').map((h) => h.trim()).filter(Boolean);
  const host = forwarded.length ? forwarded[forwarded.length - 1] : (request.headers.get('host')
?? '');
  return host.toLowerCase().split(':')[0]; // without the port
}

export function proxy(request: NextRequest) {
  if (visitorHost(request) === 'app.example.com') {
    const url = request.nextUrl.clone();
    url.pathname = `/app-area${url.pathname}`; // an internal route that serves the second address
    return NextResponse.rewrite(url);
  }
  return NextResponse.next();
}

export const config = { matcher: ['/((?!_next/static|_next/image).*)'] };
```

To test it on your own computer, add the names to the dev server's address. Browsers send `*.localhost` to your own machine:

```
curl.exe -s -o NUL -w "%{http_code}`n" -H "Host: localhost:3000" -H "X-Forwarded-Host:
app.example.com" http://127.0.0.1:3000/
```

And on the live site, check it can't be fooled. A visitor-sent header shouldn't switch addresses:

```
curl.exe -sI -H "X-Forwarded-Host: app.example.com" https://example.com/
```

That should still give your main site.

Caching: App Hosting's CDN keeps cached pages per address. Pages that differ by address should still never be cached for long. Next.js marks its prebuilt pages cacheable for a long time, so give anything the second address serves `Cache-Control: no-store` (or `no-cache`). If the second address ever served the wrong page and you fix it, check it in a private window after the next rollout.

---

10. Deploying Changes from Now On

You can release a change in two ways. Use either one, or both:

	Push to GitHub (10.1)	**Release from Antigravity** (10.2)
What gets built	the latest commit on `main`, on GitHub	the folder on your computer, as it is now
What starts it	a push, by itself	you, or Antigravity's agent, running `firebase deploy`
What you need	GitHub Desktop	Google Antigravity IDE and the Firebase CLI

Either way, App Hosting builds your site in the cloud in the same way, and the newest rollout becomes the live site. Section 10.3 shows how the two ways fit together.

Before each release, run your build and tests locally. A broken build never replaces the live site: the rollout fails, and the previous one keeps serving.

10.1 Push to GitHub

With automatic rollouts on (Section 6), **every push to `main` deploys.**

1. Change your code, and commit in GitHub Desktop.
2. Press **Push origin**.
3. App Hosting starts a rollout within a minute or so. Watch the backend's **Rollouts** tab, or the check beside the commit on GitHub.

If you work on a separate branch, it reaches the live site only when it's merged into `main` and pushed (5.2, step 2).

10.2 Release from Antigravity

Google Antigravity IDE is Google's code editor with an AI agent built in. From it, you can send the folder on your computer straight to App Hosting with the Firebase CLI, or ask the agent to do it. It goes to the same backend, so your domains (Section 7) keep working.

Set up once

1. **Open your project's folder** in Antigravity: the same folder you published in Section 5.
2. **Open a terminal** in Antigravity: press `Ctrl+`` (`Cmd+`` on a Mac).
3. **Install the Firebase CLI**, and check its version. Firebase's steps for Antigravity ask for 15.10.0 or later:

```
npm install -g firebase-tools
firebase --version
```

4. **Sign in.** Run `firebase login`, and sign in, in the browser that opens, with the Google account that owns your Firebase project. Then run `firebase projects:list`: your project should be in the list.
5. **Link the folder to your backend:**

   ```
   firebase init apphosting
   ```

   Answer its questions:

   - **Project:** choose **Use an existing project**, and pick yours.
   - **Create or link:** choose **Link to an existing backend**, and pick the backend from Section 6 (for example `my-site`). Don't create a new one: your domains are on this one.
   - **Root directory:** `/`, the same as in Section 6.
   - **"File apphosting.yaml already exists. Overwrite?":** answer **No**, the default. Yes replaces your settings from 4.5 with a blank template.

   This writes two small files: `firebase.json`, which says which backend to deploy to, and `.firebaserc`, which says which project. Neither holds a secret, so commit both.

The companion guide, Building Firebase Apps in Google Antigravity IDE, covers this setup in more detail in its Sections 1 and 2. That includes install problems on Windows, and giving the agent Firebase's skills and tools.

Deploy from the terminal

```
firebase deploy --only apphosting
```

- **The first time,** the CLI says your backend is linked to a repository on GitHub, and asks **"Are you sure you want to deploy your local source?"** Answer **Yes**. It saves your answer in `firebase.json` (`"alwaysDeployFromSource": true`), so it won't ask again.
- **Watch it** in the terminal, and in the backend's **Rollouts** tab, where the new rollout appears. A deploy from your folder takes about as long as one from GitHub.
- **`--only apphosting` sends just the site.** A plain `firebase deploy` also deploys everything else that `firebase.json` lists, such as Firestore rules.

What gets sent:

- **Your folder as it is on your disk,** including changes you haven't committed or pushed.
- **Not the files your `.gitignore` lists.** The `.gitignore` from 5.1 keeps `.env` files and keys out of the upload, too.
- **Your `apphosting.yaml`,** so the build gets the same settings as a rollout from GitHub.

The first deploy from your computer, in a region, sets up a storage bucket for the uploads. That needs a project **Owner**. If you created the project, that's you. After that, Editors can deploy too.

Or ask the agent

Antigravity's agent can run the same deploy for you. It knows Firebase's steps best with Firebase's agent skills installed (see the companion guide's Section 2).

1. Open the **Agent** pane. If it's hidden, click the **Toggle Agent** icon at the top of the window.
2. Ask it to publish, and name your backend, so it doesn't make a new one:

   > *Publish my app to my existing App Hosting backend `my-site`. Don't create a new backend.*

3. The agent asks before it runs `firebase deploy`. Read the command it shows you, then choose **Yes**.
4. Check the result in the backend's **Rollouts** tab, not only in the chat.

Keep Antigravity set to ask you before it runs commands. A setting that runs every command without asking (**Turbo**, or **Always Proceed**) also lets the agent change your live site without asking.

Start a rollout from GitHub by hand

This builds a branch or a commit that's already on GitHub, without a new push. It's useful when automatic rollouts are off, or to rebuild the live site from the same commit.

- **In the Firebase console:** open **Hosting & Serverless**, then **App Hosting**, and press **View** on your backend. Choose **Create rollout**, pick the branch (`main`), then pick the latest commit, or an earlier one by its ID. Press **Create**.
- **From Antigravity's terminal:**

  ```
  firebase apphosting:rollouts:create my-site --git-branch main
  ```

  For an earlier commit, use `--git-commit` and the commit's ID instead. With neither, it asks you which branch.

A rollout like this builds what's on GitHub, not what's in your folder. Push first.

10.3 How the two ways fit together

The newest rollout is the live site, whichever way it started. So:

- **A push replaces a deploy from your folder.** With automatic rollouts on, the next push to `main` rolls out what's on GitHub. Anything you deployed from your folder, but never pushed, disappears from the live site.
- **After a deploy from your folder, commit and push the same change.** Then GitHub matches the live site, and the next push keeps your change.
- **A deploy from your folder changes no settings.** Your backend stays linked to GitHub, and automatic rollouts stay as they were.

Should automatic rollouts stay on?

- **Yes, if pushing is how you mostly release.** It's the simplest: every push releases, and every live version is a commit on GitHub. Use Antigravity for the odd release you need before you push, then push the same change straight after.
- **No, if you want to choose when each release happens.** Pushes then only store your code on GitHub. You release with **Create rollout**, from GitHub, or with `firebase deploy --only apphosting`, from your folder.

To change it: open your backend, then **Settings**, then **Deployment**, and turn automatic rollouts off (or back on). The same page changes the live branch.

Which to use when:

You want to	Use
Release every change, with nothing extra to do	A push to `main`, with automatic rollouts on (10.1)
Release from your computer, before you push or without pushing	`firebase deploy --only apphosting`, or the agent (10.2); then commit and push
Release at a time you choose	Automatic rollouts off, then **Create rollout** when you're ready
Rebuild the live site without a new commit	**Create rollout**, with the latest commit
Undo a bad release	A rollback (10.4)

10.4 Undo a bad release

- **Fastest: roll back.** Open the backend's **Rollouts** tab. Open the three-dot menu on an earlier rollout, choose **Roll back to this build**, and confirm. It reuses that build as it was, without building again.
- **Or rebuild an older commit.** Choose **Create rollout**, then **Earlier commit**, and enter the commit's ID. This builds the old code again, with today's settings and secrets.

Then fix or revert the code on GitHub too. With automatic rollouts on, the next push to `main` rolls out again, on top of your rollback.

---

11. GitHub Actions on a Private Repository

Skip this unless you run GitHub Actions workflows (automated jobs) in your site's repository.

11.1 Watch the minutes

On GitHub's free plan, **public** repositories' Actions are free. **Private** repositories get a monthly allowance of minutes, 2,000 at the time of writing, and **each run counts as at least a whole minute**.

A scheduled job that runs every 15 minutes is 96 runs a day, about **2,880 minutes a month**: over the free allowance, even if each run takes ten seconds.

Schedule	Runs a month	Minutes used (at least)
every 15 minutes	~2,880	~2,880
every 30 minutes	~1,440	~1,440
hourly	~720	~720
daily	~30	~30

Better than a frequent schedule: have the event itself start the job. If what you're waiting for happens in **another repository**, for example a release published in a public repository, that repository can tell yours the moment it happens.

11.2 Let one repository wake another (repository_dispatch)

In your private repository, the receiving workflow listens for a named event, with a daily check as a safety net and a manual button:

```
on:
  repository_dispatch:
    types: [upstream-release]
  schedule:
    - cron: '17 6 * * *'   # once a day, in case a ping is missed
  workflow_dispatch:        # a "Run workflow" button in the Actions tab
```

Treat the ping as a wake-up call only. Have the job read whatever it needs from the source itself, rather than trusting anything the ping carries.

In the other repository, a workflow sends the ping:

```
name: Tell the site

on:
  release:
    types: [published]

permissions: {}

jobs:
  notify:
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
      - name: Wake the site's workflow
        env:
          TOKEN: ${{ secrets.SITE_DISPATCH_TOKEN }}
        run: |
          curl -fsS -X POST \
            -H "Accept: application/vnd.github+json" \
            -H "Authorization: Bearer $TOKEN" \
            -H "X-GitHub-Api-Version: 2022-11-28" \
            https://api.github.com/repos/OWNER/SITE-REPO/dispatches \
            -d '{"event_type":"upstream-release"}'
```

- **Trigger it only on events you control**, like `release`. Never `pull_request_target`, or anything else a stranger's fork could start: the token can write to your site's repository.
- `permissions: {}` gives the workflow's own built-in token nothing it doesn't need.
- A release published on GitHub's website, or with your own token, starts this workflow. One published by another workflow, using its built-in `GITHUB_TOKEN`, doesn't.

11.3 The token

1. On GitHub, click your picture, then **Settings**, then **Developer settings**, then **Personal access tokens**, then **Fine-grained tokens**, then **Generate new token**.

2. Fill it in:
    - **Name:** for example *Site release ping*.
    - **Expiration:** up to a year. Put a reminder in your calendar to renew it.
    - **Resource owner:** you.
    - **Repository access: Only select repositories**, then just your **site's** repository.
    - **Permissions:** under Repository permissions, **Contents: Read and write**. That's what GitHub requires to send a dispatch. "Metadata: Read" is added by itself; leave everything else alone.
3. **Generate token** and copy it. GitHub shows it **only once**.
4. In the **other** repository (the one sending the ping), open **Settings**, then **Secrets and variables**, then **Actions**, then **New repository secret**. Name it `SITE_DISPATCH_TOKEN` and paste the token.

Keep the token **only** in that secret. Don't paste it into chats, notes or files.

11.4 If your workflow commits or opens issues

A workflow that pushes to the repository or opens issues needs permission, and new personal repositories start **read-only**. In your site's repository, open **Settings**, then **Actions**, then **General**, then **Workflow permissions**, choose **Read and write permissions**, and save. Then give each workflow only what it uses, with a `permissions:` block (for example `contents: write` and `issues: write`).

Test it: in the **Actions** tab, pick the workflow, then **Run workflow**. A green run means it works.

---

12. Signing In the GitHub Command Line

The GitHub CLI (`gh`) is handy for checking runs, settings and issues without screenshots. To sign it in, in PowerShell:

```
gh auth login
```

Answer its questions:

1. **Where do you use GitHub?** GitHub.com
2. **Preferred protocol?** HTTPS
3. **Authenticate Git with your GitHub credentials?** Yes
4. **How would you like to authenticate?** Login with a web browser

It shows a one-time code (like `ABCD-1234`) and opens GitHub in your browser. Paste the code, sign in if asked, and press **Authorize**. Check it with `gh auth status`.

Useful afterwards:

```
gh run list -R OWNER/SITE-REPO -L 5
gh api repos/OWNER/SITE-REPO/actions/permissions/workflow
```

---

13. Troubleshooting

What you see	Why	What to do
Squarespace's **Create a host record** dialog	That's **Nameserver registration**, for running your own nameservers	Close it. Use **DNS**, then **Domain nameservers**, then **Use custom nameservers** (2.4)
Domain stops working after the nameserver switch	DNSSEC was still on at Squarespace	Turn DNSSEC off at Squarespace (2.3); it can take up to a day to recover
Cloudflare stuck on **Pending**	Cloudflare hasn't run its next check	Check the nameservers yourself (2.5). If the internet already shows Cloudflare's, wait for Cloudflare's email
Redirect rule warns **"may not apply to your traffic"**	The domain is still pending, or there's no orange `www` record yet	Add the `AAAA www 100::` proxied record (3.3); then deploy anyway
`www` "doesn't exist" right after activation	Cloudflare is still switching the new domain on	Wait up to half an hour
`http://www…` gives **522**	The redirect rule only matches `https://`	Turn on **Always Use HTTPS** (3.1)
"Too many redirects"	The encryption mode is **Flexible**, with an orange-cloud record pointing at your host	Use **Full** or **Full (strict)**, and keep Firebase's records grey (3.2)
Firebase: **"DNS changes not yet detected"** for some records	Firebase cached "nothing here" from before you added them, for about 30 minutes	Check the records answer correctly (7.4), then wait. After an hour, remove and re-add the domain in Firebase
A record's name ends in `.example.com.example.com`	The full name was typed into Cloudflare's **Name**	Edit the Name to just the part before your domain
Certificate never finishes **minting** (give it up to 24 hours)	The `_acme-challenge` CNAME is wrong or orange, a stray AAAA record is on the name, or a CAA record blocks the issuers	Recheck the CNAME (grey, exact target); remove other A/AAAA/CNAME records on the name; allow `letsencrypt.org` and `pki.goog` in any CAA record (7.2, 7.5)
The second address shows the **main site**	The code reads `Host`, but App Hosting puts the domain in `X-Forwarded-Host`	Read `X-Forwarded-Host`'s last entry, falling back to `Host` (Section 9)
GitHub Desktop says **no local changes** to publish	Publishing sends commits, not files	Press **Publish repository** (or **Push origin**); check the **History** tab
A **new, empty** repository was published	It was created in a new folder instead of adding your project's	Publish your project's own folder (5.2); the empty one can be removed
The rollout **fails**	The build broke	Open the rollout's build log; reproduce with `npm ci` and `npm run build` locally (Section 6)
`firebase` isn't found in Antigravity's terminal	The terminal was open before you installed the CLI. On macOS or Linux, your shell setup may be in `~/.bashrc`, which Antigravity's terminal doesn't load	Open a new terminal, or restart Antigravity. On macOS or Linux, load `~/.bashrc` from `~/.bash_profile`; Firebase's Studio migration page shows the lines

What you see	Why	What to do
`firebase init apphosting` asks to **overwrite `apphosting.yaml`**	It offers a blank template in place of your file	Answer **No**. If you said Yes, get your file back: in GitHub Desktop's **Changes** tab, select it and choose **Discard changes**
`firebase deploy` says **Skipping deployment of backend(s)**	You once answered No to deploying your local source, and `firebase.json` saved `"alwaysDeployFromSource": false`	Change it to `true`, or delete that line to be asked again (10.2)
`firebase deploy` also changed your **Firestore rules**	A plain `firebase deploy` deploys everything `firebase.json` lists	Use `firebase deploy --only apphosting` (10.2)
The first deploy from your computer fails with **permission denied** (403)	The first deploy from source in a region sets up a storage bucket, which needs a project Owner	Run it once while signed in (`firebase login`) as an Owner. After that, Editors can deploy too
The agent created a **second backend**	It didn't find yours	Name your backend in the prompt (10.2). Run `firebase apphosting:backends:list`, and delete only the new one, with `firebase apphosting:backends:delete`. Your domains stay on the old one
A change you deployed from Antigravity is **gone** from the live site	A push to `main` rolled out what's on GitHub, which doesn't have the change	Commit and push the change too (10.3)
`apphosting:rollouts:create` doesn't take `--git_branch`	Firebase's CLI reference table spells it with an underscore; the command's option is `--git-branch`	Use `--git-branch` (or `--git-commit`), with hyphens (10.2)
Google sign-in: **domain not authorized**	Your domain isn't in Authentication's authorized domains	Add `example.com` (4.3)
Sign-in fails only on your own computer	Since April 2025, new projects don't authorize `localhost` by default	Add `localhost` to the authorized domains (4.3)
Google sign-in or a sign-in link fails with a vague error, or `auth/operation-not-allowed`	That sign-in method isn't turned on, or Authentication's **Get started** was never pressed	Turn on the provider in **Sign-in method**; for links, **Email/Password** and **Email link** both (4.3)
A sign-in link fails with `auth/unauthorized-continue-uri`	The page the link returns to is on a domain that isn't authorized	Add the domain to **Authorized domains** (4.3)
A sign-in link says it's **invalid** or **expired**	Links work once and expire	Send a new link
The server treats a signed-in visitor as signed out	App Hosting's account lacks a role your server code needs	Read the backend's logs for the permission error, and grant that role (8.2)
A private repository's Actions run out of minutes	A frequent schedule, with each run billed as a whole minute	Run less often, or wake the job with `repository_dispatch` (Section 11)

What you see	Why	What to do
`curl` in PowerShell asks for parameters or errors oddly	In Windows PowerShell, `curl` is a different command	Use `curl.exe`
You want email at your domain later	The domain says "no email" (`v=spf1 -all`, DMARC reject)	When you set up email (for example Cloudflare Email Routing), replace those records in the same go, as your email provider says

---

14. Sources

Menus move from time to time; if one looks different, the page will show its current name.

Squarespace

- Review, change, or reset your domain's nameservers
- Nameserver registration (glue records)
- DNSSEC for Squarespace domains

Cloudflare

- Full setup: add a domain and change nameservers
- Domain statuses (Pending, Active)
- Proxy status (Proxied and DNS only)
- Create DNS records (including originless setups, `100::`)
- Redirect www to root
- Create a redirect rule in the dashboard
- Page Rules migration (template names)
- Redirecting a domain
- Always Use HTTPS
- Encryption modes

Firebase

- Add Firebase to your web app (create a project, register an app)
- Learn more about Firebase projects (the config is public)
- Firebase API keys
- App Hosting costs (Blaze plan)
- Budget alerts
- Avoid surprise bills
- Spend caps
- Password authentication
- Google sign-in

- Email link sign-in (and authorized domains)
- Redirect sign-in best practices
- Cloud Firestore quickstart
- Firestore locations
- Get started with App Hosting
- Configure App Hosting (`apphosting.yaml`)
- App Hosting frameworks and tooling (the build)
- About App Hosting (the GitHub connection)
- Connect a custom domain to App Hosting
- App Hosting troubleshooting
- Manage rollouts and releases (automatic rollouts, Create rollout, rollbacks)
- Alternative ways to deploy to App Hosting (deploying from your folder)
- Firebase CLI reference (install, `firebase login`, `firebase.json`, App Hosting commands)
- Firebase agent skills
- Firebase Studio sunset and project migration (publishing from Antigravity, the Antigravity terminal)
- Firebase CLI source, for its exact prompts: `firebase init apphosting`, deploying your local source (`alwaysDeployFromSource`), `apphosting:rollouts:create`
- App Hosting updates, July 2024 (wildcard domains and `X-Forwarded-Host`)
- Google Cloud Certificate Manager: DNS authorizations (the `_acme-challenge` record)

Google Antigravity

- Download (Antigravity IDE)
- Firebase Studio migration (publishing with the agent)
- Agent side panel
- Agent skills
- Agent settings (when the agent asks before running commands)
- Terminal sandbox

GitHub

- Add a repository from your computer to GitHub Desktop
- Add an existing project to GitHub using GitHub Desktop
- Committing and reviewing changes in GitHub Desktop (discarding changes)
- GitHub Actions billing
- Actions runner pricing (rounding to whole minutes)
- Managing personal access tokens
- Create a repository dispatch event
- Permissions required for fine-grained personal access tokens
- Using secrets in GitHub Actions

- Managing GitHub Actions settings for a repository (workflow permissions)
- `gh auth login`
- GitHub CLI quickstart

Latest version: github.com/ImmuneMoon/Connecting-a-Custom-Domain-to-Firebase-App-Hosting

If this helped, you can support Fulllion Creative Works through PayPal or Ko-fi.

© 2026 Fulllion Creative Works. All rights reserved.